LABORATORY WORK #3
TASK 1

```
CREATE DATABASE advanced_lab;

CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department VARCHAR(50) DEFAULT 'Unassigned',
    salary INTEGER,
    hire_date DATE,
    status VARCHAR(20) DEFAULT 'Active'
);

CREATE TABLE departments (
    dept_id SERIAL PRIMARY KEY,
    dept_name VARCHAR(50),
    budget INTEGER,
    manager_id INTEGER
);

CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,
    project_name VARCHAR(100),
    dept_id INTEGER,
    start_date DATE,
    end_date DATE,
    budget INTEGER
);



INSERT INTO employees
    (first_name, last_name, department, salary, hire_date, status)
VALUES
    ('Aigerim', 'Sadykova', 'IT', 90000, '2019-05-10', 'Active'),
    ('Nurlan', 'Bekov', 'IT', 65000, '2021-03-15', 'Active'),
    ('Dana', 'Omarova', 'Sales', 45000, '2023-06-20', 'Inactive'),
    ('Yerlan', 'Tulegenov', 'Sales', 82000, '2018-09-01', 'Active'),
    ('Saule', 'Akhmetova', 'HR', 50000, '2022-02-10', 'Active'),
    ('Marat', 'Ispanov', NULL, 38000, '2024-01-15', 'Inactive');

INSERT INTO departments (dept_name, budget, manager_id)
VALUES
    ('IT', 200000, 1),
    ('Sales', 150000, 2),
    ('HR', 80000, 3);

INSERT INTO projects
    (project_name, dept_id, start_date, end_date, budget)
VALUES
    ('Website Development', 1, '2025-01-01', '2025-12-01', 80000),
    ('Sales Campaign', 2, '2025-02-01', '2025-11-01', 45000),
    ('HR System', 3, '2024-01-01', '2024-12-01', 60000);



TASK 2: INSERT WITH COLUMN SPECIFICATION

INSERT INTO employees
    (emp_id, first_name, last_name, department)
VALUES
    (100, 'Ali', 'Muratov', 'IT');
```

TASK 3: INSERT WITH DEFAULT VALUES

```
INSERT INTO employees
    (first_name, last_name, salary, hire_date, department)
VALUES
    ('Aruzhan', 'Kim', 55000, CURRENT_DATE, DEFAULT);

INSERT INTO employees
    (first_name, last_name, department, hire_date)
VALUES
    ('Dias', 'Serik', 'HR', CURRENT_DATE);
```

TASK 4: INSERT MULTIPLE ROWS IN SINGLE STATEMENT

```
INSERT INTO departments (dept_name, budget, manager_id)
VALUES
    ('Marketing', 120000, 4),
    ('Finance', 180000, 5),
    ('Research', 250000, 6);
```

TASK 5: INSERT WITH EXPRESSIONS

```
INSERT INTO employees
    (first_name, last_name, department, salary, hire_date)
VALUES
    ('Timur', 'Asanov', 'IT', 50000 * 1.1, CURRENT_DATE);
```

TASK 6: INSERT FROM SELECT (SUBQUERY)

```
CREATE TEMPORARY TABLE temp_employees AS
SELECT *
FROM employees
WHERE department = 'IT';
```

TASK 7: UPDATE WITH ARITHMETIC EXPRESSIONS

```
UPDATE employees
SET salary = salary * 1.10;
```

TASK 8: UPDATE WITH WHERE CLAUSE AND MULTIPLE CONDITIONS

```
UPDATE employees
SET status = 'Senior'
WHERE salary > 60000
  AND hire_date < DATE '2020-01-01';
```

TASK 9: UPDATE USING CASE EXPRESSION

```
UPDATE employees
SET department = CASE
    WHEN salary > 80000 THEN 'Management'
    WHEN salary BETWEEN 50000 AND 80000 THEN 'Senior'
    ELSE 'Junior'
END;
```

TASK 10: UPDATE WITH DEFAULT

```
UPDATE employees
SET department = DEFAULT
WHERE status = 'Inactive';
```

TASK 11: UPDATE WITH SUBQUERY

```
UPDATE departments d
SET budget = (
    SELECT COALESCE(AVG(e.salary), 0) * 1.20
    FROM employees e
    WHERE e.department = d.dept_name
)
WHERE EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department = d.dept_name
);
```

TASK 12: UPDATE MULTIPLE COLUMNS

```
UPDATE employees
SET salary = salary * 1.15,
    status = 'Promoted'
WHERE department = 'Sales';
```

TASK 13: DELETE WITH SIMPLE WHERE CONDITION

```
DELETE FROM employees
WHERE status = 'Terminated';
```

TASK 14: DELETE WITH COMPLEX WHERE CLAUSE

```
DELETE FROM employees
WHERE salary < 40000
  AND hire_date > DATE '2023-01-01'
  AND department IS NULL;
```

TASK 15: DELETE WITH SUBQUERY

```
DELETE FROM departments d
WHERE NOT EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department = d.dept_name
);
```

TASK 16: DELETE WITH RETURNING CLAUSE

```
DELETE FROM projects
WHERE end_date < DATE '2023-01-01'
RETURNING *;
```

TASK 17: INSERT WITH NULL VALUES

```
INSERT INTO employees
```

```
    (first_name, last_name, salary, department)
VALUES
    ('Test', 'Null', NULL, NULL);


TASK 18: UPDATE NULL HANDLING

UPDATE employees
SET department = 'Unassigned'
WHERE department IS NULL;


TASK 19: DELETE WITH NULL CONDITIONS

DELETE FROM employees
WHERE salary IS NULL
   OR department IS NULL;


TASK 20: INSERT WITH RETURNING

INSERT INTO employees
    (first_name, last_name, department, salary, hire_date)
VALUES
    ('Madina', 'Aliyeva', 'IT', 70000, CURRENT_DATE)
RETURNING emp_id, first_name || ' ' || last_name AS full_name;


TASK 21: UPDATE WITH RETURNING

WITH updated AS (
    SELECT emp_id, salary AS old_salary
    FROM employees
    WHERE department = 'IT'
      AND salary IS NOT NULL
)
UPDATE employees e
SET salary = e.salary + 5000
FROM updated u
WHERE e.emp_id = u.emp_id
RETURNING e.emp_id, u.old_salary, e.salary AS new_salary;


TASK 22: DELETE WITH RETURNING ALL COLUMNS

DELETE FROM employees
WHERE hire_date < DATE '2020-01-01'
RETURNING *;


TASK 23: CONDITIONAL INSERT

INSERT INTO employees
    (first_name, last_name, department, salary, hire_date)
SELECT
    'Ayan', 'Nurlanov', 'IT', 60000, CURRENT_DATE
WHERE NOT EXISTS (
    SELECT 1
    FROM employees
    WHERE first_name = 'Ayan'
      AND last_name = 'Nurlanov'
);
```

TASK 24: UPDATE WITH JOIN LOGIC USING SUBQUERIES

```
UPDATE employees e
SET salary = ROUND(
    e.salary * CASE
        WHEN d.budget > 100000 THEN 1.10
        ELSE 1.05
    END
)
FROM departments d
WHERE e.department = d.dept_name
  AND e.salary IS NOT NULL;
```

TASK 25: BULK OPERATIONS

```
INSERT INTO employees
    (first_name, last_name, department, salary, hire_date)
VALUES
    ('Employee1', 'Test1', 'IT', 40000, CURRENT_DATE),
    ('Employee2', 'Test2', 'Sales', 42000, CURRENT_DATE),
    ('Employee3', 'Test3', 'HR', 44000, CURRENT_DATE),
    ('Employee4', 'Test4', 'IT', 46000, CURRENT_DATE),
    ('Employee5', 'Test5', 'Sales', 48000, CURRENT_DATE);

UPDATE employees
SET salary = salary * 1.10
WHERE first_name IN (
    'Employee1', 'Employee2', 'Employee3',
    'Employee4', 'Employee5'
);
```

TASK 26: DATA MIGRATION SIMULATION

```
CREATE TABLE employee_archive (
    LIKE employees INCLUDING ALL
);

WITH moved AS (
    DELETE FROM employees
    WHERE status = 'Inactive'
    RETURNING *
)
INSERT INTO employee_archive
SELECT *
FROM moved;
```

TASK 27: COMPLEX BUSINESS LOGIC

```
UPDATE projects p
SET end_date = p.end_date + 30
WHERE p.budget > 50000
  AND p.end_date IS NOT NULL
  AND (
      SELECT COUNT(*)
      FROM employees e
      JOIN departments d
        ON e.department = d.dept_name
      WHERE d.dept_id = p.dept_id
  ) > 3;
```